```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
import numpy as np
```

Q1. Load and Freeze VGG16

```
# Loading the VGG16 model using pretrained weights (weights='imagenet') with include_top=False
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
print("VGG16 base model loaded.")
```

```
VGG16 base model loaded.
```

```
# Freezing all layers of the model.
for layer in base_model.layers:
    layer.trainable = False
print("All layers in the base model have been frozen.")
```

```
All layers in the base model have been frozen.
```

```
# Printing the number of trainable and non-trainable parameters in the model.
# As we know TensorFlow/Keras models have built-in ways to count parameters.
total_params = base_model.count_params()
trainable_params = np.sum([tf.keras.backend.count_params(w) for w in base_model.trainable_weights])
non_trainable_params = np.sum([tf.keras.backend.count_params(w) for w in base_model.non_trainable_weights])
```

```
print("\nParameter Counts for the VGG16 Base Model:")
print(f"  Total parameters:{total_params:,}")
print(f"  Trainable parameters:{trainable_params:,}")
print(f"  Non-trainable parameters:{non_trainable_params:,}")
```

```

Parameter Counts for the VGG16 Base Model:
  Total parameters:14,714,688
  Trainable parameters:0.0
  Non-trainable parameters:14,714,688
```

```
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.vgg16 import preprocess_input as preprocess_input_vgg16
from tensorflow.keras.applications.resnet50 import preprocess_input as preprocess_input_resnet50
import matplotlib.pyplot as plt
```

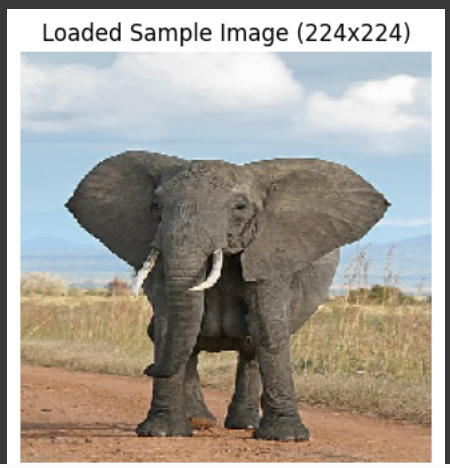
Q2. Compare Output Shapes of VGG16 and ResNet50

```
img_url = 'https://upload.wikimedia.org/wikipedia/commons/thumb/3/37/African_Bush_Elephant.jpg/330px-African_Bush_Elephant.jpg'
img_path = get_file('elephant.jpg', origin=img_url)
print(f"Sample image downloaded to: {img_path}")

# Loading the image and resizing it to 224x224
img = image.load_img(img_path, target_size=(224, 224))
```

```
Sample image downloaded to: /root/.keras/datasets/elephant.jpg
```

```
# Loaded image:-
plt.figure(figsize=(4,4))
plt.imshow(img)
plt.title("Loaded Sample Image (224x224)")
plt.axis('off')
plt.show()
```



```
# Convert this Image to a numpy array
img_array = image.img_to_array(img)
print(f"Loaded image array shape: {img_array.shape}")
```

```
Loaded image array shape: (224, 224, 3)
```

```
# Adding batch dimension
img_batch = np.expand_dims(img_array, axis=0)
```

```
print("\n Processing with VGG16...")
# Preprocessing for VGG16
img_preprocessed_vgg = preprocess_input_vgg16(np.copy(img_batch))
print(f" Shape after VGG16 preprocessing: {img_preprocessed_vgg.shape}")
```

```

 Processing with VGG16...
 Shape after VGG16 preprocessing: (1, 224, 224, 3)
```

```
#Passing it through VGG16 (with include_top=False).
vgg16_base = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
features_vgg16 = vgg16_base.predict(img_preprocessed_vgg)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 749ms/step
```

```
# 4) Printing the output feature map shape from VGG16.
print(f"  Output feature map shape from VGG16: {features_vgg16.shape}")
```

```
  Output feature map shape from VGG16: (1, 7, 7, 512)
```

```
print("\n Processing with ResNet50...")
# Preprocessing for ResNet50
img_preprocessed_resnet = preprocess_input_resnet50(np.copy(img_batch))
print(f"  Shape after ResNet50 preprocessing: {img_preprocessed_resnet.shape}")
```

```

 Processing with ResNet50...
  Shape after ResNet50 preprocessing: (1, 224, 224, 3)
```

```
# Passing it through ResNet50 (with include_top=False).
resnet50_base = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
features_resnet50 = resnet50_base.predict(img_preprocessed_resnet)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
```

```
# Printing the output feature map shape from ResNet50.
print(f"  Output feature map shape from ResNet50: {features_resnet50.shape}")
```

```
  Output feature map shape from ResNet50: (1, 7, 7, 2048)
```

Q3. Use ResNet50 for Classification

```
from tensorflow.keras.applications.resnet50 import preprocess_input as resnet50_preprocess_input
from tensorflow.keras.applications.resnet50 import decode_predictions as resnet50_decode_predictions
from tensorflow.keras.preprocessing import image
from tensorflow.keras.utils import get_file
```

```
#Loading the ResNet50 model with include_top=True and pretrained ImageNet weights.
model_resnet_full = ResNet50(weights='imagenet', include_top=True)
# No input_shape needed here as we are using the default (224,224,3) with the top.
print("ResNet50 full model loaded.")
```

```
ResNet50 full model loaded.
```

```
#Loading a sample image, resize and preprocess it correctly.
img_url = 'https://upload.wikimedia.org/wikipedia/commons/thumb/3/37/African_Bush_Elephant.jpg/330px-African_Bush_Elephant.jpg'
img_path = get_file('elephant2.jpg', origin=img_url)
print(f"Sample image downloaded to: {img_path}")
```

```
Sample image downloaded to: /root/.keras/datasets/elephant2.jpg
```

```
# Loading the image and resizing it
img = image.load_img(img_path, target_size=(224, 224))
```

```
# Displaying the loaded image
plt.figure(figsize=(4,4))
plt.imshow(img)
plt.title("Sample Image for ResNet50 Classification")
plt.axis('off')
plt.show()
```

Sample Image for ResNet50 Classification

```
# Converting the image to a numpy array
img_array = image.img_to_array(img)
```

```
# Preprocessing the image:
# Adding batch dimension
img_batch = np.expand_dims(img_array, axis=0)
img_preprocessed = resnet50_preprocess_input(np.copy(img_batch))
print(f"Image preprocessed. Shape: {img_preprocessed.shape}")
```

```
Image preprocessed. Shape: (1, 224, 224, 3)
```

```
# Using the model to predict the class.
print("\nPredicting class with ResNet50...")
predictions_raw = model_resnet_full.predict(img_preprocessed)
print("Prediction complete.")
```

```

Predicting class with ResNet50...
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
Prediction complete.
```

```
# Displaying the top 3 predictions using decode_predictions.
decoded_preds = resnet50_decode_predictions(predictions_raw, top=3)[0]
print("\nTop 3 Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_preds):
    print(f"{i+1}: {label} ({score:.2f})")
```

```
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Top 3 Predictions:
1: African_elephant (0.95)
2: tusker (0.03)
3: Indian_elephant (0.02)
```